

BSc and MSc Student Project Portfolio

Industrial projects, graph applications, self-supervised time series, vision models and signal processing

How to use this portfolio

The portfolio contains fifteen projects in five categories. Each entry states the task, data source and access conditions, starting papers and code, required baselines, evaluation protocol, Bachelor scope and possible Master contribution.

Bachelor projects focus on implementation, comparison and one focused ablation. **Master projects** add a stated research question, a method or evaluation change, multiple runs and a detailed analysis. A controlled negative result is acceptable when the evaluation answers the stated question.

Department of Electrical and Computer Engineering
Aarhus University

Supervisor: Naveed

Project scope and evaluation

What counts as a BSc or MSc contribution

Dimension	Bachelor project	Master project
Starting point	One dataset and one established method	Verified anchor-paper reproduction plus required baselines
Contribution	Documented pipeline, fair comparison, one focused ablation	New mechanism, evaluation protocol, graph/view construction or transfer/robustness study
Evidence	Correct split, two or three baselines, error analysis	Multiple seeds, uncertainty, matched budgets, ablations and resource reporting where relevant
Outcome	Explain whether and when the method helps	Answer the stated research question; a controlled negative result is valid

Portfolio index

Compute estimates assume compact models, moderate batch sizes and no large hyperparameter sweep. Projects marked as requiring a preliminary check need a data or licence review before work begins.

ID	Project	Suggested level	Compute
1.1	Siemens Gamesa: Data-driven operational state identification for wind turbine condition monitoring	MSc preferred	Light to moderate
1.2	Siemens Gamesa: Probabilistic drivetrain configuration and kinematics identification from vibration signals	MSc preferred	Light to moderate
1.3	Tetra Pak: Regime-aware self-supervised learning for non-stationary filtration cycles	MSc preferred	Light to moderate
2.1	Remaining useful life prediction for aircraft engines	BSc or MSc	Very light to light
2.2	Multi-horizon traffic forecasting with data-driven graph structure	BSc or MSc	Light to moderate
2.3	Illicit Bitcoin transaction detection	BSc or MSc	Light to moderate
2.4	Graph-based movie recommendation with accuracy and coverage	BSc only	Very light
2.5	Molecular HIV activity prediction with calibrated GNNs	BSc or MSc	Light to moderate
3.1	Contrastive learning for wearable activity recognition	BSc only	Light
3.2	Lead-aware masked learning for 12-lead ECG	MSc preferred	Moderate
3.3	Self-supervised forecasting representations for energy series	BSc or MSc	Light to moderate
3.4	Self-supervised learning on heat-pump smart-meter data	MSc preferred	Light to moderate after preprocessing
3.5	Graph self-supervised learning from vibration signals	MSc only	Moderate preprocessing
3.6	Time–frequency contrastive Mamba for cross-condition bearing diagnosis	MSc preferred	Light to moderate
3.7	Correlation-graph Mamba for order-robust multivariate forecasting	MSc preferred	Light to moderate on Weather
4.1	Adapting DINO backbones for industrial anomaly detection	BSc or MSc	Light for frozen features
4.2	Adapting DINOv3 for robust abdominal CT segmentation	BSc or MSc	Moderate for BSc and moderate to high for MSc
4.3	Federated knowledge distillation for medical image denoising	MSc only	Moderate
5.1	Deep-unfolded multivariate VMD for correlated multichannel signals	MSc only	Light for synthetic tests

How to choose and run a project

A simple decision process

1. Choose the **task**, not the fashionable model: node classification, link prediction, graph prediction, temporal forecasting or representation learning.
2. Download the data and run one complete baseline on a small subset before the formal project begins.
3. Write the research question as one sentence and identify the comparison that could falsify it.
4. Freeze the split, primary metric and compute budget before extensive development.
5. Treat the authors' repository as an executable specification—not as the thesis contribution.

Shared protocol for self-supervised and pretrained projects

Every self-supervised or pretrained project should report the following settings whenever they are meaningful:

- **Frozen linear evaluation:** pretrain without labels, freeze the encoder and train only a linear or otherwise explicitly lightweight head.
- **Fine-tuning:** initialise from the pretrained encoder and update it with labels.
- **Fully supervised baseline:** train the same encoder architecture from scratch using labels.
- **Random-initialisation control:** freeze a randomly initialised encoder and train the same head. Without this, the value of pretraining is ambiguous.
- **Label-efficiency curve:** report several budgets, typically 1%, 5%, 10% and 100%, rather than one full-label result.

Leakage rule

Group splits by the real independent unit: subject, patient, household, engine, bearing or process cycle—never by window. On graphs, state whether pretraining is transductive (evaluation nodes/graphs visible without labels) or strict inductive (held out entirely). On time series, use chronological splits where the task is forecasting.

Report ten seeds for small citation networks and at least three elsewhere. Report results even when they do not beat a leaderboard; the evaluation should answer the stated question.

Licences and access

Resource	Licence or status	Practical consequence
UCI HAR	CC BY 4.0	Open with attribution
PTB-XL	CC BY 4.0	No credentialing required
ETT	Open research dataset	Check the repository terms before redistribution
MovieLens 100K	GroupLens research terms	Academic use; cite the dataset
OGB ogbg-molhiv	Open, MIT-licensed tooling	Use the official split and evaluator
Elliptic Bitcoin	Distributor's research terms	Confirm terms before publishing results
Paderborn KAt	CC BY-NC 4.0	Non-commercial academic use
MVTec AD and MVTEC AD 2	CC BY-NC-SA 4.0	Non-commercial and share-alike
DINOv2	Apache 2.0	Ungated
HEAPO data	Public download; no dataset licence stated on Zenodo	Obtain written permission for thesis use; the dataloader is MIT-licensed
LargeST	CC BY-NC 4.0; benchmark code MIT	Non-commercial academic use with attribution
Tetra Pak process data	Partner-supplied under an approved agreement	Restricted access; confirm scope and secure environment before allocation
Siemens Gamesa fleet data	Partner-supplied; subject to the project data agreement	Restricted access; confirm scope and secure environment before allocation

Category 1 Industrial projects: machine learning + signal processing

These projects are developed with Siemens Gamesa and Tetra Pak using non-public operational data. At the start of each project, the student, supervisor and partner will confirm the available data, downstream task and permitted outputs.

All three are primarily Master projects. A reduced Bachelor version can be offered when a prepared dataset and fixed task are available. The Bachelor project implements and compares established methods; the Master project adds a new representation, feature design or evaluation question.

Evaluation split. Keep all measurements from one turbine or filtration cycle in the same split. A held-out site or production line can be used as an additional Master-level transfer test when enough data are available.

Learn first and find ideas

- Operational-state discovery: [TS2Vec](#), clustering with [scikit-learn](#), and the [UMAP documentation](#) for visualisation.
- Time-series classification: [MiniRocket](#), [InceptionTime](#) and the [sktime toolkit](#).
- Representation learning and non-stationarity: [TS2Vec](#), [TS-TCC](#) and [SimMTM](#).

Project 1.1 Siemens Gamesa: Data-driven operational state identification for wind turbine condition monitoring

Task	Unsupervised operating-state discovery followed by condition-monitoring evaluation
Recommended level	MSc preferred; reduced BSc version possible
Compute	Light to moderate; compact models on a 4–8 GB GPU
Data	Partner-supplied Siemens Gamesa vibration and operational data; not public
Starting paper	Yue et al., TS2Vec , AAAI 2022

What you will do. Wind-turbine monitoring systems often group vibration measurements by power output. However, turbines producing the same power can show different vibration behaviour. You will test whether operating states learned from vibration and operational measurements describe the turbine more consistently than those learned from power bins.

Project goal. Discover recurring operating states and determine whether using those states improves a downstream condition-monitoring model. The project should answer whether the learned states contain useful information beyond turbine power. Where possible, the discovered operating states should be interpreted in relation to known turbine operating regimes, controller actions, or structural and aerodynamic phenomena.

Data. Siemens Gamesa will provide anonymised turbine data under an agreed access arrangement. The data may include vibration features or spectra together with variables such as power, rotor or generator speed, pitch angle, torque, wind conditions and controller information. The exact channels, number of turbines, time period and sampling format must be confirmed before allocation.

Methods and code. Start with power binning, PCA and one clustering method such as k-means or a Gaussian mixture. Then test one representation-learning approach, such as [TS2Vec](#), followed by clustering. Use UMAP only for visualisation, not as evidence that the states are valid.

Baselines and evaluation. Compare power bins, clustering on standardised operational variables, and clustering on the learned representation. Evaluate whether states are stable across held-out turbines or time periods and whether they have a clear operational interpretation. The main downstream comparison uses the same condition-monitoring or anomaly model with power-bin conditioning and learned-state conditioning. Report residual variation and, if suitable event or alarm labels are available, detection performance and false-alarm rate.

Practical notes. Unsupervised clusters are not automatically physical operating states. Do not judge the result only by silhouette score or a two-dimensional plot. A useful state should be repeatable, physically interpretable where possible, and should improve an agreed downstream measure.

Bachelor scope

Bachelor version. Using a prepared feature table, compare power binning with one clustering method. Describe the resulting states using the available operational variables and measure how much within-state vibration variation each grouping removes. Representation learning is not required.

Master scope

Master research question. **Do data-driven operating states describe wind-turbine vibration more consistently than power bins, and do they improve downstream condition monitoring?** The contribution is the state representation and a controlled comparison showing when it improves—or does not improve—the downstream model. Include stability analysis and one ablation of the input variables or representation method.

Project 1.2 Siemens Gamesa: Probabilistic drivetrain configuration and kinematics identification from vibration signals

Task	Probabilistic ranking of candidate drivetrain configurations from vibration and operational measurements
Recommended level	MSc preferred; reduced BSc version possible
Compute	Light to moderate; compact models on a 4–8 GB GPU
Data	Partner-supplied Siemens Gamesa data with configuration information for a subset of turbines; not public
Starting paper	Dempster, Petitjean and Webb, ROCKET , DMKD 2020

What you will do. A wind-turbine fleet can contain several bearing, gearbox or drivetrain variants, and the monitoring system may not always know which configuration is installed. Because component geometry affects the vibration spectrum, you will test whether measured vibration and operational data can narrow the set of plausible configurations and provide probabilities or confidence estimates for the candidates.

Project goal. Use vibration and operational measurements to determine whether fixed drivetrain hardware variants can be distinguished and, when candidate configurations are known, rank the most plausible configurations for each turbine. Verified configuration information for a subset of turbines or components will be used to validate and calibrate the rankings. As an optional physics-informed extension, condition the model on known bearing or drivetrain kinematics and test whether relationships learned from well-documented components can be transferred to more complex components with limited labels.

Data. Siemens Gamesa will provide anonymised vibration and operational measurements, potentially with verified configuration information for a subset of turbines. Before allocation, confirm the number of turbines and configuration classes, how the configuration labels were obtained, the measurement positions, sampling format, and whether raw waveforms, spectra or extracted features are available. The probabilistic ranking study requires enough independent turbines for each candidate configuration.

Method and code. Extract spectral features from the vibration measurements and combine them with available operational data. Train a simple probabilistic model, such as [multinomial logistic regression](#), to assign a probability to each candidate drivetrain configuration. If raw spectra or time series are available, compare it with [MiniRocket](#), for which [open-source code](#) is available. As a physics-informed extension, include available bearing or drivetrain kinematic information in the model and test whether it improves the configuration ranking, particularly for components with limited labelled data.

Evaluation. Compare models using operational data only, vibration features only, and both together, and evaluate them on turbines not used for training. Assess whether the correct configuration is ranked first or among the leading candidates and whether the reported confidence is reliable.

Bachelor scope

Bachelor version. Using a prepared feature table and verified configuration labels, compare one probabilistic classifier with the metadata-only baseline on turbine-grouped splits. Report both the highest-ranked classification result and whether the true configuration appears among the top candidate configurations. Identify the most informative spectral or operational features.

Master scope

Master research question. Can vibration and operational measurements narrow the set of plausible drivetrain configurations and provide calibrated confidence estimates beyond what operational metadata already reveals? The contribution may be a calibrated candidate-ranking method, a physics-informed feature set or a model conditioned on kinematic information. Include a metadata-only control and analyse performance across operating conditions.

Project 1.3 Tetra Pak: Regime-aware self-supervised learning for non-stationary filtration cycles

Task	Self-supervised multivariate time-series learning followed by cycle-level regression or anomaly detection
Recommended level	MSc preferred; reduced BSc version possible
Compute	Light to moderate; compact models on a 4–8 GB GPU
Data	Partner-supplied Tetra Pak filtration-process data; not public
Starting paper	Yue et al., TS2Vec , AAAI 2022

What you will do. Membrane-filtration plants generate repeated production and cleaning cycles containing flow, pressure, temperature and energy measurements. Because operating conditions differ between cycles, you will learn representations that account for this variation and test them on one agreed prediction or anomaly-detection task.

Project goal. Determine whether cycle-phase- and operating-regime-aware self-supervised learning separates normal process variation from inefficient or abnormal behaviour better than a generic time-series representation.

Data. Tetra Pak will provide anonymised process data or access through a secure environment. Possible variables include cycle boundaries, flow, pressure, temperature, power or energy, setpoints and operating conditions. The number of independent cycles, sampling rate, available plants or lines, and downstream target must be confirmed before allocation. Split by complete cycle and preserve time order.

Methods and code. Use one established method—[TS2Vec](#), [TS-TCC](#) or [SimMTM](#)—with a compact 1-D CNN, TCN or GRU encoder. Compare generic augmentations with a cycle-phase- or regime-aware view or masking design. Change-point detection or clustering may be used to describe operating regimes.

Baselines and evaluation. Compare a statistical baseline, a supervised TCN or GRU, generic self-supervised learning and the proposed regime-aware version. Include random-frozen and frozen-probe controls. Use MAE for early prediction of final specific energy, or AUPRC for an agreed anomaly label. Report a label-efficiency curve and use several seeds when the number of independent cycles permits.

Practical notes. Confirm the independent-cycle count and one measurable downstream target before allocation. Rerunning an unchanged self-supervised method is a baseline, not the Master contribution. Follow the [shared protocol for self-supervised projects](#).

Bachelor scope

Bachelor version. Using pre-segmented filtration cycles and a fixed downstream target, compare one statistical or supervised baseline with one established self-supervised method. Use cycle-grouped splits and evaluate a frozen representation with a small prediction head. Designing a new augmentation or representation is not required.

Master scope

Master research question. Can cycle-phase- and operating-regime-aware self-supervised views improve prediction or anomaly detection across non-stationary filtration cycles? The contribution is the process-aware view or masking design and its matched comparison with generic augmentations, including transfer to a held-out regime, line or product when the data support it.

Category 2 GNN applications for real-world data

A **graph** represents entities and the relationships between them. The entities are **nodes** and their relationships are **edges**. Both may carry information: a customer node might contain age or location, a product node might contain category and price, and an edge might record that a customer bought or rated a product. Road sensors connected by roads, transactions linked by payments and atoms connected by chemical bonds are other examples.

A **graph neural network (GNN)** learns numerical representations, or embeddings, by combining a node's own features with information from its neighbours. In a customer-product graph, the model can use previous purchases, product properties and patterns from nearby customers to score a new customer-product link. The same principle supports **node classification**, **link prediction** and **graph-level prediction**.

Each project below asks a specific question about when and why the graph helps, not simply whether a GNN can be trained. Students define what the nodes and edges mean, fix a split and a primary metric, and include a control that tests whether the graph contributes.

Application project requirements

Specify a defensible graph construction, a fixed split and primary metric, a non-graph baseline and at least one GNN. Include an edge-removal, edge-randomisation or alternative-graph experiment to test whether the graph contributes.

Learn first and find ideas

- Begin with [A Gentle Introduction to Graph Neural Networks](#), then use the [Stanford CS224W course and project guidance](#) and the [CS224W project archive](#).
- [PyTorch Geometric dataset catalogue](#), [Open Graph Benchmark](#), [SNAP datasets](#) and the [Temporal Graph Benchmark](#).
- Stanford examples: [traffic as a dynamic graph](#), [water quality with RGCNs](#), [fake-news detection](#) and [NFT transaction prediction](#).

Project 2.1 Remaining useful life prediction for aircraft engines

Task	Graph-level temporal regression
Recommended level	BSc or MSc
Compute	Very light to light; CPU or 4 GB GPU
Data	NASA C-MAPSS – FD001: 100 training engines, 100 test engines, 21 sensor channels
Starting paper	Huang, He and Sick, Spatio-Temporal Attention GNN for RUL , 2024

What you will do. Predict how many operating cycles remain before a turbofan engine fails. Each sensor channel becomes a node and edges express relationships among sensors. The project's real subject is the graph itself: does it matter how you connect the sensors?

Data. Start with FD001; FD002–FD004 are optional transfer sets. Reserve complete training engines for validation and use the official test set for final scores. Split by **engine**, never by window. NASA provides final RUL labels for test engines, so early/middle/late-life analysis belongs on the held-out validation engines. The download is public; NASA does not specify a licence on the catalogue page.

Methods and code. Use a GRU or TCN as the graph-free baseline and a compact GCN–GRU or GAT–GRU as the graph model. The [GNN RUL benchmark](#) is a useful reference; a small [PyTorch Geometric](#) implementation is sufficient.

Baselines and evaluation. Compare a constant/degradation-stage baseline, one sequence model and one compact GNN. Use MAE or RMSE as primary, and report three seeds for MSc work.

Practical notes. Fix the RUL cap, normalisation and metric before modelling; changing them makes published results incomparable.

Bachelor scope

Bachelor research question. How sensitive is GNN-based remaining-useful-life prediction to the way the sensor graph is constructed?

Core experiment. Keep the model fixed and compare correlation, sensor-similarity, random and no-edge graphs, plus the graph-free sequence baseline.

Interpretation. FD001 has only 21 sensor channels, so the graph may add little beyond temporal modelling. A correlation graph performing no better than a random graph is a valid result.

Master scope

Master research question. Can a sparse, time-varying sensor graph improve accuracy when sensors are noisy or missing? Develop one graph-learning mechanism, compare it with fixed and random graphs at matched capacity, and add one robustness or cross-subset transfer study.

Project 2.2 Multi-horizon traffic forecasting with data-driven graph structure

Task	Node-level temporal regression
Recommended level	BSc or MSc
Compute	Light to moderate; 4–8 GB GPU
Data	METR-LA — 207 sensors, 34,272 five-minute observations; PEMS-BAY optional
Starting paper	Wu et al., Graph WaveNet , IJCAI 2019

What you will do. Forecast traffic speed at every road sensor 15, 30 and 60 minutes ahead, and determine when the road network's structure actually helps. Short-horizon forecasts may depend mostly on a sensor's own recent history, while longer horizons may require information propagating through the network.

Data. Use METR-LA with the standard chronological 70/10/20 split. Fit scalers and learned/correlation graphs on training data only. The files are publicly downloadable through DCRNN, but no separate dataset licence is stated there; do not redistribute them.

Methods and code. Compare historical average and a graph-free GRU with [Graph WaveNet](#) or a compact STGCN. DCRNN is optional because its reference implementation is slower and uses an old TensorFlow stack.

Baselines and evaluation. Use historical average, graph-free GRU, and the same GNN with distance, randomised and identity/no-message-passing graphs. Report masked MAE (primary), RMSE and MAPE at 15, 30 and 60 minutes.

Practical notes. Profile early and use top-k or low-rank adjacency; a dense graph at every time step is unnecessarily expensive.

Bachelor scope

Bachelor research question. Does graph information become more valuable as the forecasting horizon increases?

Core experiment. With one trained 12-step model, compare the distance, randomised and identity/no-message-passing graphs at steps 3, 6 and 12 (15, 30 and 60 minutes). Include the graph-free GRU as a separate baseline.

Note. Evaluate one 12-step model at different horizons; do not train three separate models.

Master scope

Master research question. Does a sparse, traffic-regime-conditioned adjacency improve 60-minute forecasts over road and learned static graphs? Match model capacity, use three seeds, and analyse whether edges change plausibly between rush hour and free flow.

Project 2.3 Illicit Bitcoin transaction detection

Task	Node classification on a temporal graph
Recommended level	BSc or MSc
Compute	Light to moderate; the full graph fits on a 4–8 GB GPU
Data	Elliptic Bitcoin dataset via PyG — 203,769 nodes, 234,355 edges, 165 model features, 49 time steps
Starting paper	Weber et al., Anti-Money Laundering in Bitcoin , 2019

What you will do. Classify labelled Bitcoin transactions as licit or illicit, then test how missing payment edges affect the result.

Data. The PyG loader downloads the data and defines time steps 1–34 as training and 35–49 as test; take validation steps only from 1–34. Unknown labels are **unlabelled**, not negatives. The dataset is CC BY-NC-ND 4.0: academic analysis is allowed, but commercial use and redistribution of modified data are restricted.

Methods and code. Compare logistic regression or random forest with GCN and GraphSAGE in [PyTorch Geometric](#). Use class weighting only after the unweighted baseline is established.

Baselines and evaluation. Use the first 93 PyG features for the truly local, graph-free baseline. Report a second 165-feature baseline separately because the last 72 features already aggregate neighbours. Compare both with GCN/GraphSAGE and a no-edge control. Use AUPRC or illicit-class F1 as primary and report performance by time step.

Practical notes. The 49 time steps are disconnected snapshots. Preserve time order and keep label handling and threshold selection identical across methods.

Bachelor scope

Bachelor research question. How robust are GNN-based illicit-transaction detectors when graph information is incomplete?

Core experiment. Delete 0%, 10%, 25% and 50% of test edges and compare both GNNs with the 93-feature local baseline. Repeat deletion with several random seeds.

Interpretation. A 165-feature random forest is a strong baseline partly because 72 features already summarise neighbours. Keep it, but do not call it graph-free.

Master scope

Master research question. Can a temporally calibrated GNN remain useful under concept drift and missing edges?

Add one temporal or uncertainty mechanism and evaluate it by time step and at a fixed alert budget.

Project 2.4 Graph-based movie recommendation with accuracy and coverage

Task	Link prediction and ranking on a bipartite graph
Recommended level	BSc only
Compute	Very light; CPU or low-memory GPU
Data	MovieLens 100K — 100,000 ratings, 943 users, 1,682 movies, 5 MB
Starting paper	He et al., LightGCN , SIGIR 2020

What you will do. Rank unseen films for each user and test whether better ranking accuracy reduces catalogue coverage or long-tail exposure.

Data. Treat ratings of 4–5 as positive interactions. Split each user's positives chronologically and remove validation/test edges from the training graph. Evaluate over the full unseen catalogue. The download is public for research, but its licence forbids redistribution and commercial use without permission.

Methods and code. Compare popularity, BPR matrix factorisation and LightGCN. A direct [PyTorch Geometric](#) implementation is simpler than adapting the [official LightGCN repository](#).

Baselines and evaluation. Use identical training negatives for BPR-MF and LightGCN. Report NDCG@20 or Recall@20, catalogue coverage and average recommended-item popularity.

Practical notes. Avoid future-interaction leakage and sampled-negative evaluation. Submit a reproducible pipeline, controlled comparison and error analysis.

Bachelor scope

Bachelor research question. Does graph-based recommendation improve ranking accuracy at the expense of catalogue coverage and long-tail exposure?

Core experiment. Compare the three methods, then vary LightGCN depth or its training negative sampler and plot the accuracy/coverage trade-off. Use at least three seeds and analyse results by user activity and item popularity.

Project 2.5 Molecular HIV activity prediction with calibrated GNNs

Task	Graph classification under distribution shift
Recommended level	BSc or MSc
Compute	Light to moderate; 4–8 GB GPU
Data	OGB ogbg-molhiv — 41,127 molecular graphs, official scaffold split and evaluator
Starting paper	Hu et al., Open Graph Benchmark , NeurIPS 2020

What you will do. Predict HIV-assay activity and test whether a GNN generalises to unseen molecular scaffolds better than chemical fingerprints.

Data. The [OGB loader](#) automatically downloads the MIT-licensed data and supplies the scaffold split and evaluator. Do not compare scaffold-split results with random-split figures.

Methods and code. Compare Morgan fingerprints with logistic regression or random forest via [RDKit](#) against GCN and edge-aware GINE models. OGB and [PyTorch Geometric](#) provide loaders and baseline components.

Baselines and evaluation. Use a fingerprint classifier, GCN and GINE, with one bond-feature ablation. Report ROC-AUC (official primary metric), AUPRC and three seeds.

Practical notes. Keep the official split and evaluator fixed; choose thresholds and calibration parameters on validation data only.

Bachelor scope

Bachelor research question. Do GNNs generalise better than traditional molecular fingerprints to scaffolds not seen during training?

Core experiment. Compare fingerprints, GCN and GINE on the official split, then inspect errors on scaffolds most dissimilar to training data.

Interpretation. A fingerprint/tree model may match or beat a plain GNN. Select all hyperparameters on validation data; never tune against the test split.

Master scope

Master research question. Can calibration improve on held-out scaffolds without reducing ROC-AUC? Compare temperature scaling with one ensemble or uncertainty method and report calibration by scaffold novelty.

Category 3 Self supervised representation learning for time series

A **time series** is an ordered sequence of measurements. A wearable device may record acceleration every fraction of a second, an ECG contains voltage measurements from several leads, and an industrial plant records pressure, flow, temperature and energy over time. These signals contain repeated patterns, trends and operating phases, but useful labels may require expert work and are often scarce.

Self-supervised learning (SSL) learns from the unlabelled recordings by creating its own training task. A model might make two valid views of the same signal agree, hide part of a sequence and predict it, or predict the representation of a future segment. The resulting encoder converts a recording or window into a compact numerical representation. A small classification or regression head is then trained for a downstream task.

Evaluation focuses on whether the representation reduces the amount of labelled data needed and transfers to new people, machines or operating conditions. Each project compares SSL with a fully supervised model and a randomly initialised encoder on leakage-safe splits.

Three ideas to understand

Contrastive learning aligns valid views of the same recording. Masked modelling reconstructs hidden samples or patches. Latent prediction estimates hidden representations instead of raw values and requires explicit collapse monitoring. Possible MSc contributions include view or mask designs based on the physics or physiology of the signal.

Learn first and find ideas

- Methods with code: [TS-TCC](#), [TS2Vec](#), [TF-C](#) and [SimMTM](#).
- Data sources: [UCI Machine Learning Repository](#), [PhysioNet](#), [UCR/UEA archive](#), [Monash forecasting archive](#) and [Zenodo](#).
- Monitor embedding variance and covariance from the first epoch in latent-predictive work. A smooth pretraining loss does not establish that the representation is useful.

Project 3.1 Contrastive learning for wearable activity recognition

Task	Self-supervised time-series representation learning, then classification
Recommended level	BSc only
Compute	Light; CPU or 4 GB GPU
Data	UCI Human Activity Recognition — 30 subjects, 6 activities, 10,299 windows, 50 Hz, CC BY 4.0
Starting paper	Eldele et al., TS-TCC , IJCAI 2021

What you will do. A phone records acceleration and angular velocity while a person walks, sits or climbs stairs. Learn an encoder from the unlabelled sensor windows, freeze it, and measure how few labels a small classifier needs to recognise activities.

Data. UCI HAR is a 58 MB public download under CC BY 4.0. It contains pre-windowed 2.56-second recordings with 50% overlap and a standard subject-independent split. Keep that split.

Methods and code. [TS-TCC](#) or [TS2Vec](#) — choose **one**. Use the same 1-D CNN or small Transformer encoder for the self-supervised and supervised comparisons.

Baselines and evaluation. Random frozen encoder, SSL frozen linear probe, SSL fine-tuning and a fully supervised model trained from scratch. Report accuracy and macro-F1 on the subject-independent split.

Bachelor scope

Bachelor research question. How much labelled activity data can self-supervised pretraining replace?

Core experiment. Compare all four conditions at 1%, 5%, 10%, 25% and 100% label budgets. Resample the labelled windows within the training subjects and report confidence intervals. Optionally test robustness by jointly rotating the accelerometer and gyroscope axes.

Project 3.2 Lead-aware masked learning for 12-lead ECG

Task	Self-supervised multivariate time-series learning, then multi-label classification
Recommended level	MSc preferred; BSc with a supplied data pipeline
Compute	Moderate; 8 GB GPU recommended
Data	PTB-XL — 21,799 ECGs, 18,869 patients, 100/500 Hz, CC BY 4.0, no credentialing
Starting paper	Dong et al., SimMTM , NeurIPS 2023

What you will do. A 12-lead ECG observes the same cardiac event from different electrical viewpoints. Instead of masking arbitrary time patches, the model hides complete leads or anatomical lead groups and predicts their latent representations from the visible leads.

Data. PTB-XL at 100 Hz, using the official patient-aware folds: 1–8 for training, 9 for validation, 10 for testing. The dataset is fully labelled, so label budgets simulate scarcity rather than reflecting missing annotations.

Methods and code. Start with [SimMTM](#) or [TS2Vec](#) and a 1-D ResNet or compact Transformer. The [PTB-XL benchmark code](#) provides preprocessing and supervised references.

Baselines and evaluation. Random frozen encoder, frozen probe, fine-tuning and the same encoder fully supervised. Macro-AUROC on fold 10 across the five diagnostic superclasses is the primary endpoint; add macro-AUPRC for class imbalance. Report patient-level splits and several label budgets.

Bachelor scope

Bachelor question. How does diagnostic performance change when fewer ECG leads are available? With the supplied pipeline and one encoder, compare 12, 6 and 3 leads and lead II alone under supervised training and one established SSL method.

Master scope

Master research question. Does lead-aware masking improve low-label diagnosis over random temporal masking? Develop complete-lead or anatomical-group masking, condition the predictor on lead identity, and monitor embedding variance for collapse. Ablate masking granularity and predictor depth.

Project 3.3 Self-supervised forecasting representations for energy series

Task	Self-supervised time-series learning, then temporal regression
Recommended level	BSc or MSc
Compute	Light to moderate; 4–8 GB GPU
Data	ETT — electricity transformer temperature, hourly (ETTh1/ETTh2) and 15-minute (ETTm1/ETTm2) subsets
Starting paper	Woo et al., CoST , ICLR 2022

What you will do. Pretrain an encoder on one unlabelled energy series, freeze it, and fit a small forecasting head. Test whether the representation transfers to a different series better than training a supervised forecaster on that series.

Data. Use ETT with chronological splits and training-only scaling. ETTh1 and ETTh2 come from different transformers. The public repository uses CC BY-ND 4.0; cite it and do not redistribute modified copies.

Methods and code. [CoST](#), [TS2Vec](#) or [SimMTM](#) for pretraining. [DLinear](#) is the required supervised reference; [PatchTST](#) is optional. A ridge or linear head on frozen representations is the key diagnostic.

Baselines and evaluation. Seasonal-naive, DLinear, frozen SSL representation with a ridge head, and SSL fine-tuning. Report MSE and MAE at agreed horizons with identical input lengths and chronological splits, plus three seeds.

Bachelor scope

Bachelor research question. Do self-supervised representations pretrained on one energy series transfer to a different one?

Core experiment. Pretrain on ETTh1, then probe and fine-tune on ETTh2. Compare with direct ETTh2 pretraining and DLinear trained on ETTh2.

Interpretation. DLinear may outperform the SSL pipeline. Analyse whether the gap changes with forecast horizon and target-data volume.

Master scope

Master research question. Do decomposition-aligned positive pairs improve transfer and low-data forecasting when seasonal phase drifts? Use a stable seasonal decomposition to select positives, compare with random cropping and frequency-domain contrast, and analyse non-stationary intervals. Validate the period estimate on ETT before adding irregular data.

Project 3.4 Self-supervised learning on heat-pump smart-meter data

Task	Time-series representation learning, then household-level classification
Recommended level	MSc preferred; BSc metadata-label version
Compute	Light to moderate after preprocessing
Data	HEAPO on Zenodo — 1,408 Swiss households, 15-minute and daily resolution, 2018–2024; dataset licence must be confirmed
Starting paper	Brudermueller et al., HEAPO , 2025

What you will do. Pretrain on household electricity series, then test whether the representations improve prediction of a household attribute or an inspection-derived issue.

Data. The 458 MB public download contains 1,408 households; 214 have both smart-meter data and inspection information. Use the [official MIT-licensed dataloader](#). The Zenodo Rights field does not state a dataset licence, so confirm reuse terms before allocation. Group all splits by household.

Methods and code. Use [TS2Vec](#) or [SimMTM](#), with a 1-D CNN or InceptionTime supervised baseline. Use heat-pump type for the lower-risk task, or one inspection-derived issue only after counting usable positives among the 214 matched households.

Baselines and evaluation. Compare random-frozen, frozen SSL, fine-tuned SSL and supervised encoders on identical household splits. Include weather-only and metadata-only baselines. Report balanced accuracy, macro-F1 and confidence intervals using repeated grouped cross-validation.

Bachelor scope

Bachelor version. Use heat-pump type or another well-covered metadata label. Compare supervised training with one SSL encoder on household-grouped splits and report a label-efficiency curve.

Master scope

Master research question. Can unlabelled data from a large related household population improve prediction on a much smaller labelled population, and do weather- and cycle-aware views improve it further? Compare pretraining sources — none, labelled households only, unlabelled control households, and all households — then add outdoor-temperature-matched positives or whole-cycle masking and ablate the weather conditioning.

Feasibility condition. An inspection-label study has only 214 matched households. Select one primary label and verify its prevalence before committing to it; otherwise use the better-covered heat-pump-type task.

Project 3.5 Graph self-supervised learning from vibration signals

Task	Graph-level classification (one graph per signal window) with cross-operating-condition transfer
Recommended level	MSc only
Compute	Moderate preprocessing; small models on a 4–8 GB GPU
Data	Paderborn KAt — ~5 GB, 32 bearings, four operating conditions, CC BY-NC 4.0
Starting paper	Hou et al., GraphMAE , KDD 2022 — official code

What you will do. Convert each vibration window into a graph of fault-related frequency bands, pretrain a graph encoder without labels, and test transfer to a different load or speed condition.

Graph construction. Resample each window into the shaft-order domain. Use fixed nodes for BPFO, BPFI, BSF and FTF bands and their harmonics. Node features can include band energy, envelope amplitude, spectral kurtosis and RMS; edges can represent envelope correlation or spectral coherence. Fit thresholds on training data only.

Data. Paderborn provides 64 kHz vibration, speed, torque and radial load for 32 physical bearings under four operating conditions. Download it from the [official archive](#). Split by bearing, not by window. CWRU may be used only to develop the preprocessing pipeline; its official site does not state a clear data licence.

Methods and code. Use a GIN encoder with [GraphMAE](#). Compare with supervised GIN and a 1-D CNN. Do not remove fault-frequency nodes as a generic graph augmentation.

Baselines and evaluation. Compare a 1-D CNN, envelope-spectrum features with gradient boosting, supervised GIN, random-frozen GIN, and GraphMAE with frozen and fine-tuned heads. Test physics-based, similarity, random and no-edge graphs. Report accuracy and macro-F1; cross-condition transfer is primary.

Practical notes. The dataset has only 32 independent bearings. Use bearing-grouped splits, report confidence intervals and keep the graph, SSL method and transfer experiment fixed in advance. This project is not suitable for a short BSc project.

Master scope

Master research question. Does a speed-invariant graph built from bearing kinematics transfer across operating conditions better than a similarity graph, and does self-supervised pretraining add value once the graph is fixed? The contribution is the graph construction and controlled ablation; GraphMAE is a baseline.

Project 3.6 Time–frequency contrastive Mamba for cross-condition bearing diagnosis

Task	Self-supervised time-series representation learning, then fault classification
Recommended level	MSc preferred; BSc with the processed dataset and a fixed comparison
Compute	Light to moderate; 4–8 GB GPU
Data	Paderborn KAt – 32 bearings, four operating conditions, 64 kHz vibration, CC BY-NC 4.0
Starting paper	Zhang et al., TF-C , NeurIPS 2022 – official code

What you will do. Use Mamba as the temporal encoder in TF-C, which learns jointly from time- and frequency-domain views. Test whether the resulting representation transfers between bearing operating conditions when only a small labelled set is available.

Data. The official Paderborn archive contains vibration, motor current, speed, torque and radial load for six healthy, twelve artificially damaged and fourteen naturally damaged bearings. Begin with the processed FD-A and FD-B subsets provided through the TF-C repository. For the final evaluation, group the split by physical bearing or recording so overlapping windows cannot occur in both training and test sets.

Methods and code. Start from the [TF-C implementation](#) and replace its time-domain CNN or Transformer with a compact [Mamba](#) encoder. Retain the time–frequency contrastive objective and use a linear or small multilayer classification head.

Baselines and evaluation. Compare the original TF-C encoder, supervised 1-D CNN, supervised Mamba, random-frozen Mamba, frozen TF-C–Mamba and fine-tuned TF-C–Mamba. Report macro-F1, balanced accuracy, per-class recall and at least three seeds. The primary test is transfer to a different operating condition; report several labelled-example budgets and separate results for previously observed and unseen bearings where the data permit.

Practical notes. The public code and processed data provide a complete starting pipeline, but the older TF-C dependencies may need updating. Test the Mamba environment during the first project week. The raw data are several gigabytes and are needed only for the MSc signal-processing extensions.

Bachelor scope

Bachelor research question. Under the same TF-C objective, does a compact Mamba encoder improve cross-condition classification or computational efficiency over the existing encoder? Use the processed FD-A and FD-B data, fixed augmentations and identical splits. Compare accuracy, macro-F1, training time and memory without proposing a new pretraining objective.

Master scope

Master research question. Can bearing-specific time–frequency views improve the transfer of Mamba representations across operating conditions and unseen bearings? Develop one process-informed view, such as an envelope-spectrum view centred on bearing fault frequencies, an order-normalised view or a positive pair drawn from the same bearing under different conditions. Compare it with the original TF-C augmentations and ablate the view construction.

Project 3.7 Correlation-graph Mamba for order-robust multivariate forecasting

Task	Multivariate time-series forecasting
Recommended level	MSc preferred; BSc evaluation version
Compute	Light to moderate on Weather; moderate on Electricity
Data	Weather and Electricity – public forecasting benchmarks with 21 and 321 channels
Starting paper	Cai et al., MambaTS – MIT-licensed code

What you will do. Represent each signal channel as a graph node, exchange information between correlated channels, and then use a shared Mamba encoder to model each node through time. Test whether explicit channel structure improves forecasting and reduces sensitivity to arbitrary channel order.

Data. Develop the method on Weather, which has 21 meteorological variables and 52,696 ten-minute observations. Confirm the result on Electricity, which contains 321 hourly electricity-demand series and 26,304 observations. Use chronological splits and fit scaling, correlations and graph thresholds on the training period only. The datasets and scripts are available through [Autoformer](#); [OpenTS](#) documents their dimensions and standard protocol.

Methods and code. Use [MambaTS](#) as the main code base and [S-Mamba](#) and [C-Mamba](#) as channel-modelling references. Build a sparse top- k correlation graph from the training data. At each time step or patch, apply a shared GCN, GraphSAGE or GAT layer across channels; then apply the same temporal Mamba encoder to every node and a shared forecasting head.

Baselines and evaluation. Compare seasonal naive, [DLinear](#), channel-independent Mamba, MambaTS and one existing channel-aware Mamba model. Graph controls must include identity, random and shuffled-correlation adjacency. Report MSE and MAE at standard horizons, three seeds, parameter count, training time and memory. Consistently permute the input channels, graph and outputs to measure order sensitivity, and mask channels at test time to measure robustness.

Practical notes. Use Weather for development and a sparse graph for Electricity. Correlation describes statistical association rather than a physical or causal link. A consistent relabelling of nodes, adjacency and outputs should not materially change a correctly implemented shared graph-Mamba model.

Bachelor scope

Bachelor research question. How sensitive are existing Mamba forecasters to channel order, and do their channel-interaction mechanisms improve over channel-independent Mamba? Reproduce MambaTS or S-Mamba on Weather and compare channel-independent and channel-aware variants under channel permutations and missing-channel tests. A new graph architecture is not required.

Master scope

Master research question. Can a sparse correlation-graph channel mixer improve forecasting and reduce sensitivity to arbitrary variable ordering compared with existing Mamba channel-mixing methods? Implement the graph module, match model capacity, and ablate graph type, sign handling, sparsity and message-passing depth on Weather and Electricity. A window-dependent graph is an optional extension after the fixed-graph study.

Category 4 Deep-learning vision models and applications

A vision foundation model is an image encoder pretrained on a large image collection and reused for later tasks. Vision Transformers divide an image into patches and learn a feature vector for each patch. Self-supervised models such as DINO learn these features without class labels.

Students do not train a foundation model from scratch. In industrial anomaly detection, a frozen encoder can compare a test image with known normal images and identify unusual patches. An adapter can instead tune the encoder to the appearance of the normal training images without using defect examples.

MVTec training sets contain normal images only; real test defects cannot be used to train or tune the model. Frozen features, parameter-efficient adaptation and full fine-tuning are comparable only when they use the same training information and objective.

Three ways to adapt a pretrained backbone

Freeze it and score distances from normal reference patches. Adapt it efficiently by training small LoRA or adapter modules. Fine-tune it by updating the complete encoder. In this project, adaptation uses a self-supervised consistency objective on normal images only.

Learn first and find data

- Core reading: [DINO](#), [DINOv2](#), the [DINOv2 model card and code](#), and [AnomalyDINO](#) with its [official code](#).
- Parameter-efficient adaptation: [LoRA](#) and the [Hugging Face PEFT library](#).
- Normal-only objectives: [DINO self-distillation](#) and [masked autoencoding](#). The proposed project uses patch-level teacher-student consistency as the simpler starting point.
- Data and model hubs: [MVTec AD](#), [Hugging Face models](#) and [timm](#).
- Keep zero-shot, one-shot, few-shot and full-shot anomaly results separate, and never compare an image-level metric directly with a pixel-level metric.

Project 4.1 Adapting DINO backbones for industrial anomaly detection

Task	Image-level anomaly detection and pixel-level anomaly localisation
Recommended level	BSc or MSc
Compute	Light for frozen features; moderate for adaptation (12–24 GB GPU recommended)
Data	MVTec AD — 5,354 images, 15 categories, pixel-precise masks, CC BY-NC-SA 4.0
Starting paper	Oquab et al., DINOv2 , 2023

What you will do. Use DINOv2 patch features to detect and locate defects from normal reference images. The BSc project studies the frozen detector. The MSc project adapts the encoder using only defect-free training images.

Data. [MVTec AD](#) contains 5,354 images in 15 categories, with defect-free training images and labelled normal and defective test images. [MVTec AD 2](#) contains 8,000+ images in eight harder scenarios and includes unseen lighting conditions. Both use CC BY-NC-SA 4.0; MVTec AD 2 also has a private-test evaluation server.

Methods and code. Use DINOv2 ViT-S/14 from the [Apache-2.0 repository](#). The frozen route follows [AnomalyDINO](#): store normal patch features and score each test patch by its distance to the memory bank. For the MSc route, keep a frozen DINOv2 teacher and train LoRA or adapters in a student model with [PEFT](#). Make patch features agree between an original normal image and a photometrically altered view of the same image. No defect image or mask is used during adaptation.

Baselines and evaluation. Compare frozen AnomalyDINO with [PatchCore](#). For the MSc route, compare frozen DINOv2, normal-only LoRA/adaptor adaptation and normal-only full fine-tuning under the same consistency loss. Report image AUROC, pixel AUROC and the official AU-PRO measure per category, plus trainable parameters, memory and runtime. Keep shot count and public/private test protocols separate.

Practical notes. DINOv2 code and standard weights are ungated and Apache 2.0. MVTec data is restricted to non-commercial use. Complete the BSc frozen baseline before beginning any adaptation.

Bachelor scope

Bachelor research question. How do the number of normal reference images and the selected DINOv2 feature layer affect defect detection and localisation?

Core experiment. Reproduce AnomalyDINO on MVTec AD with ViT-S/14. Compare 1, 4 and 16 normal reference images, two feature layers selected in advance, and PatchCore. Repeat the reference-image sampling and analyse false positives and localisation errors. No model training is required.

Master scope

Master research question. Can normal-only parameter-efficient adaptation make DINOv2 patch features robust to industrial lighting variation without removing the local differences needed to detect unseen defects? Use a frozen teacher and a LoRA/adaptor student. Train only on normal images with patch-aligned brightness, contrast and noise transformations. Compare patch-level consistency with global-image consistency, and compare frozen, LoRA/adaptor and full fine-tuning under the same loss on MVTec AD and MVTec AD 2. Ablate adapter depth and transformation strength.

The contribution is the normal-only patch-consistency method and its controlled evaluation. Monitor feature variance and distance from the original DINOv2 representation to detect collapse or over-adaptation. Training on real test defects, or repeatedly tuning choices on public test labels, would invalidate the unsupervised setting. Synthetic defects may be included only as a clearly separated optional comparison.

Project 4.2 Adapting DINOv3 for robust abdominal CT segmentation

Task	Abdominal-organ segmentation and robustness evaluation
Recommended level	BSc or MSc
Compute	Moderate for BSc and moderate to high for MSc; 16–40 GB GPU recommended
Data	AMOS22 — 500 abdominal CT volumes with voxel-level annotations of 15 organs
Starting paper	

What you will do. Use a pretrained DINOv3 vision model as the encoder of a medical-image segmentation system. The model will process axial slices from abdominal CT volumes, while a U-Net-style decoder will generate organ masks. You will compare frozen and adapted DINOv3 representations and determine how reliably they perform when image quality deteriorates.

Project goal. Determine whether features learned by DINOv3 from natural images can support abdominal-organ segmentation and whether parameter-efficient adaptation improves robustness to medical-image corruption without requiring full retraining of the foundation model.

Data. Use the CT portion of the AMOS22 dataset, which provides 3D abdominal scans with annotations for 15 organs. Convert each volume into axial slices or neighbouring-slice 2.5D inputs. All slices from one patient must remain in the same training, validation or test partition. Controlled test corruptions may include noise, blur, reduced resolution, contrast changes and intensity shifts.

Methods and code. Start with the pretrained DINOv3 ViT-S/16 backbone. Load the official weights through [Hugging Face](#) or the official DINOv3 repository. Use a compact U-Net-style decoder that combines patch features from several transformer layers. Compare this model with a conventional 2D U-Net or nnU-Net baseline. The [MedDINOv3 repository](#) provides an example of integrating DINOv3 with nnU-Net for medical segmentation.

Baselines and evaluation. Compare a conventional U-Net, frozen DINOv3 with a trainable decoder, and the selected adaptation method. Use Dice as the primary metric and report HD95 as the boundary-error metric. Report performance separately for each organ and corruption type. Also report trainable parameters, GPU memory and training time. All models must use the same patient splits and corruption protocol.

Practical notes. DINOv3 is a 2D vision model, so this project uses slice-wise or 2.5D segmentation rather than direct 3D DINOv3 processing. Training DINOv3 from scratch is outside the project scope. Begin with ViT-S/16 and a frozen encoder before attempting LoRA or partial fine-tuning. Full end-to-end fine-tuning should be treated as an optional reference experiment only if sufficient GPU resources are available.

Bachelor scope

Bachelor research question. How robust are frozen DINOv3 features for abdominal-organ segmentation when CT image quality deteriorates?

Keep the DINOv3 encoder frozen and train only the segmentation decoder. Compare frozen DINOv3 with a conventional 2D U-Net on clean images and predefined corruption levels. Analyse which organs and image conditions cause the largest performance loss. No adaptation or full backbone fine-tuning is required.

Master scope

Master research question. Can parameter-efficient adaptation improve the robustness of DINOv3 for abdominal-organ segmentation without losing performance on clean images?

Compare a frozen DINOv3 encoder, LoRA or adapter-based adaptation, and partial fine-tuning of the final transformer blocks. Use the same decoder, training data and corruption protocol for every variant. Study at least one adaptation factor, such as adapter depth, LoRA rank or the number of unfrozen transformer blocks. Report several random seeds and analyse the trade-off between segmentation performance, robustness, trainable parameters and computational cost.

Project 4.3 Federated knowledge distillation for medical image denoising

Task	Federated medical-image denoising with output-level knowledge distillation
Recommended level	MSc only
Compute	Moderate; one 12–24 GB GPU is sufficient for sequential client simulation
Data	M&Ms cardiac MRI partitioned by scanner vendor; ACDC as the shared proxy set
Starting paper	

What you will do. Simulate several hospitals that train medical-image denoising models without sharing their local scans. Each hospital trains a U-Net on its own cardiac MRI data. Instead of sending model weights to the central server, the hospitals apply their local models to the same public proxy images and send only the resulting denoised predictions. The server uses these predictions to train a global U-Net student through knowledge distillation.

Project goal. Determine whether prediction-based federated distillation can produce a denoising model that generalises across scanner vendors and noise conditions without transferring local images or local model weights to the central server.

Data. Treat scanner vendors in the M&Ms cardiac MRI dataset as simulated federated clients. Use the original images as reference targets and generate controlled noisy inputs using predefined MRI-relevant corruptions, such as Rician noise and intensity non-uniformity. Use ACDC cardiac MRI images only as a shared public proxy set for communication between clients and the server. Split by patient or volume, never by image slice, and keep the final test patients separate from local training and proxy distillation.

Methods and code. Use the same compact 2D U-Net architecture for the local teachers and the global student in the core experiment. Each client trains its local U-Net to map a noisy slice to its reference image. The clients then denoise the same corrupted proxy images and transmit only their output images or predicted noise residuals. The server aggregates these predictions and trains the global student using a reconstruction loss to the aggregated teacher target. The clients' private images and model weights remain local. Use [Flower](#) to organise the simulated clients and communication rounds; the denoising models can be implemented in PyTorch or [MONAI](#).

Baselines and evaluation. Compare local-only U-Nets, centralized training as an experimental upper bound, standard FedAvg with weight sharing, and output-level federated distillation without weight sharing. Use SSIM as the primary image-quality metric and report PSNR and NMSE as secondary metrics. Report results separately for each scanner vendor and noise condition, together with communication volume and training time. Include an unseen-noise or held-out-vendor test to measure generalization.

Practical notes. The original MRI images are reference images, not perfect noise-free ground truth. Therefore, conclusions should concern removal of the controlled synthetic corruptions rather than clinical scanner-noise correction. Prediction sharing reduces direct exposure compared with transmitting model weights, but it does not provide a formal privacy guarantee; differential privacy or secure aggregation may be discussed as optional extensions rather than assumed properties.

Master scope

Master research question. Can output-level federated distillation produce a generalizable medical-image denoising model without sharing local model weights, and how robust is it when clients have different scanner domains, noise distributions or model capacities? Compare local training, centralized training, FedAvg and federated distillation under matched data splits and noise conditions. Compare distillation of denoised images with distillation of predicted noise residuals, and study at least one additional factor such as proxy-set size, confidence-weighted teacher aggregation, heterogeneous U-Net widths or transfer to a held-out vendor. Use several seeds and report the performance–communication trade-off.

Category 5 Signal processing and deep-unfolded models

Many signal-processing methods repeat a fixed optimisation step until convergence. Algorithm unrolling, also called deep unfolding, turns a fixed number of those iterations into neural-network layers. The update equations determine the network structure, while selected algorithm parameters are learned from data.

This differs from Category 3: the network is a truncated version of a known algorithm, not a general-purpose encoder. Evaluation must therefore test both numerical correctness and the downstream task. Faster execution alone is insufficient if the learned layers no longer perform the intended decomposition.

A Master-level category

This category requires graduate-level optimisation and signal processing. The contribution is a justified change to the unrolled algorithm and a controlled comparison with the converged classical method.

Learn first and find ideas

- General unrolling: Gregor and LeCun, [Learning Fast Approximations of Sparse Coding \(LISTA\)](#), ICML 2010; Monga, Li and Eldar, [Algorithm Unrolling](#), IEEE SPM 2021; and Yang et al., [Deep ADMM-Net](#), NeurIPS 2016.
- Signal decomposition: Dragomiretskiy and Zosso, [Variational Mode Decomposition](#), with [reference Python code](#), and Rehman and Aftab, [Multivariate Variational Mode Decomposition](#).
- Ahmad et al., [Robust Spatiotemporal Forecasting Using Adaptive Deep-Unfolded VMD](#), 2025 preprint, with the [authors' repository](#). The repository currently has no licence file.

Project 5.1 Deep-unfolded multivariate VMD for correlated multichannel signals

Task	Unroll multivariate VMD and evaluate decomposition, robustness and forecasting
Recommended level	MSc only
Compute	Light for synthetic tests; moderate for LargeST-SD (12–24 GB GPU recommended)
Data	Synthetic multichannel signals plus LargeST-SD — 716 sensors, CC BY-NC 4.0
Starting paper	Ahmad, Wesemann, Waschkowski & Khalid, Adaptive Deep-Unfolded VMD , 2025

What you will do. Variational Mode Decomposition separates a signal into band-limited modes. MVMD extends it to several channels by estimating a shared centre frequency for each mode. You will turn a fixed number of MVMD iterations into trainable layers and test whether robust cross-channel weighting helps when some channels are noisy.

Project goal. Build a coupled unrolled MVMD whose output for one channel depends on the other channels. Replace equal averaging in the shared-frequency update with permutation-invariant weights derived from each channel's residual. Test whether the weights reduce the influence of corrupted channels without collapsing onto one channel.

Data. First use synthetic signals with known modes and centre frequencies, 2–32 channels, varied SNR and controlled channel corruption. Then use the **San Diego 2019 subset** of [LargeST](#): 716 traffic sensors aggregated to 15-minute intervals. The full California and Greater Los Angeles datasets are outside the project compute budget. LargeST data is CC BY-NC 4.0 and its benchmark code is MIT-licensed.

Methods and code. Validate classical MVMD using the [original MATLAB implementation](#) and an [MIT-licensed Python implementation](#). Reimplement the UVMD equations from the [anchor paper](#), then extend them to the coupled update. The authors' MAGN repository may be inspected, but its code should not be copied or modified without licence clarification.

Baselines and evaluation. Compare per-channel VMD, converged MVMD, per-channel UVMD, equal-weight unrolled MVMD and the proposed robust-weighted version. On synthetic data report reconstruction error, centre-frequency error, runtime and performance across SNR, channel count and corrupted-channel fraction. On LargeST-SD use one fixed compact forecaster for raw data and every decomposition method; report MAE and RMSE at 15, 30 and 60 minutes. Use chronological splits and at least three seeds for learned methods.

Practical notes. This is a high-risk MSc topic. In the first two weeks, recover known synthetic modes with classical MVMD, reproduce one UVMD result, and verify that one channel's output has a non-zero gradient with respect to another channel's input. Do not start LargeST until these tests pass. Regularise or constrain the channel weights and report their effective number so that apparent robustness is not produced by selecting one channel.

Master scope

Master research question. Can a fixed-depth unrolled MVMD match converged MVMD, and can residual-based channel weighting improve shared-frequency recovery when some channels are corrupted?

The minimum contribution is a correct coupled update, residual-based channel weighting, and an equal-weight ablation. Test unseen SNR levels and channel counts. LargeST-SD is a secondary test of whether decomposition improvements affect forecasting; it is not necessary to reproduce the full MAGN benchmark.

Selection guidance

Levels

The standard Bachelor options are **2.1–2.5**, **3.1** (UCI-HAR), **3.3** (ETT) and **4.1** (MVTec AD). Of these, **2.4** (MovieLens) and **3.1** are Bachelor-only; the others also support a Master scope.

Projects **3.2**, **3.4**, **3.5** and **5.1** are intended primarily for Master students. Projects **3.2** and **3.4** list reduced Bachelor versions.

All Category 1 projects are Master-preferred because the data are partner-supplied and the research task must be finalized after data access. Projects **1.1**, **1.2** and **1.3** can be offered at Bachelor level when a prepared dataset and a fixed target are supplied.

Week-one feasibility checklist

- Obtain access to the exact dataset and record its version, size, licence or data agreement, and release date or extraction period.
- Run the simplest baseline from raw input to final metric on a small subset.
- Identify the independent split unit and verify that it cannot cross train and test.
- Count how many research repositories must be adapted. Adapting one is a Bachelor project. Adapting two is a Master project and is a common reason a project runs out of time with no results.
- Check that the test set is large enough to separate the conditions being compared. If a grouped test fold contains only a few dozen units, confidence intervals may be wider than the effect under study.
- Estimate training time and peak memory before planning seeds or ablations.
- For MSc work, write the research question, proposed mechanism, main alternative explanation and minimum viable ablation table.

Recurring failure modes

- Comparing with a number produced by a different split, metric, horizon, shot count or evaluation level.
- Using windows from the same physical unit in both training and test sets.
- Comparing two published implementations and concluding something about the method when encoder, augmentation and prior tuning are confounded with the mechanism being tested.
- Reporting a difference that the test set is too small to resolve.
- Calling an authors' repository run a research contribution.
- Reporting only in-distribution performance when transfer or cold start is the motivation, or when the benchmark is already saturated.
- Omitting the random frozen encoder in a self-supervised study.
- Expanding to a second dataset before the first pipeline and evaluation protocol are stable.

Select one primary question, fix the evaluation protocol before extensive experimentation and report negative as well as positive results.

Partner-supplied projects are grouped in Category 1. Allocation follows confirmation of data access, sample count, labels or target definition, and the permitted form of thesis reporting.